

```
import { useState, useRef, useEffect } from "react";

const SYSTEM_PROMPT = `You are an elite coding assistant – deeply knowledgeable,

- Writing, debugging, and refactoring code in any language
- Explaining complex programming concepts clearly
- Reviewing code for bugs, security issues, and performance
- Architecting systems and suggesting best practices
- Answering questions about frameworks, libraries, and tools
```

When writing code:

- Always use fenced code blocks with the language tag (e.g. ````python`)
- Add concise inline comments for non-obvious logic
- Prefer idiomatic, production-quality code
- If multiple approaches exist, briefly mention tradeoffs

Be direct and confident. Lead with the answer, not preamble. If a question is amb

```
const SUGGESTED = [
  "Write a binary search in Python",
  "Explain async/await in JavaScript",
  "Debug: why does my React useEffect run twice?",
  "Build a REST API with Express.js",
];

function parseContent(text) {
  const parts = [];
  const regex = /```(\w+)?\n([\s\S]*)```/g;
  let last = 0, m;
  while ((m = regex.exec(text)) !== null) {
    if (m.index > last) parts.push({ type: "text", content: text.slice(last, m.index) });
    parts.push({ type: "code", lang: m[1] || "code", content: m[2].trimEnd() });
    last = m.index + m[0].length;
  }
  if (last < text.length) parts.push({ type: "text", content: text.slice(last) });
  return parts;
}
```

```
function CodeBlock({ lang, content }) {
  const [copied, setCopied] = useState(false);
  const copy = () => {
    navigator.clipboard.writeText(content);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };
}
```

```

    };
    return (
      <div style={{ background:"#06060e", border:"1px solid #1a1a2e", borderRadius:
        <div style={{ display:"flex", justifyContent:"space-between", alignItems:"c
          <span style={{ fontSize:11, color:"#7c6aff", fontWeight:700, letterSpacing:
            <button onClick={copy} style={{ background:copied?"#0f3d2a":"#12122a", bo
              {copied ? "✓ COPIED" : "COPY"}
            </button>
          </div>
        <pre style={{ margin:0, padding:"16px", overflowX:"auto", fontSize:13, line
      </div>
    );
  }
}

```

```

function Message({ role, parts }) {
  const isUser = role === "user";
  return (
    <div style={{ display:"flex", flexDirection:isUser?"row-reverse":"row", gap:1
      <div style={{ width:34, height:34, borderRadius:"50%", flexShrink:0, displa
        {isUser ? "👤" : "⚡"}
      </div>
      <div style={{ maxWidth:"82%", background:isUser?"#16103a":"#0c0c1e", border
        {parts.map((p, i) =>
          p.type === "code"
            ? <CodeBlock key={i} lang={p.lang} content={p.content} />
            : <div key={i} style={{ fontSize:14, lineHeight:1.8, color:isUser?"#c
          )}
        </div>
      </div>
    </div>
  );
}

```

```

export default function CodingAI() {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);
  const bottomRef = useRef(null);
  const textareaRef = useRef(null);

  useEffect(() => { bottomRef.current?.scrollIntoView({ behavior:"smooth" }); },

  const autoResize = () => {
    const ta = textareaRef.current;
    if (!ta) return;
    ta.style.height = "auto";
    ta.style.height = Math.min(ta.scrollHeight, 180) + "px";
  };
}

```

```

const send = async (text) => {
  const q = (text || input).trim();
  if (!q || loading) return;
  setInput("");
  if (textareaRef.current) textareaRef.current.style.height = "auto";

  const userMsg = { role: "user", content: q };
  const history = [...messages, userMsg];
  setMessages(history);
  setLoading(true);

  try {
    const res = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: {
        "content-type": "application/json",
        "anthropic-version": "2023-06-01",
        "anthropic-dangerous-direct-browser-access": "true",
      },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514",
        max_tokens: 1000,
        system: SYSTEM_PROMPT,
        messages: history.map(m => ({ role: m.role, content: m.content })),
      }),
    });

    if (!res.ok) {
      const err = await res.json().catch(() => ({}));
      throw new Error(err?.error?.message || `HTTP ${res.status}`);
    }

    const data = await res.json();
    const reply = data.content?.map(b => b.text || "").join("") || "No response";
    setMessages(prev => [...prev, { role: "assistant", content: reply }]);
  } catch (e) {
    setMessages(prev => [...prev, { role: "assistant", content: `⚠️ ${e.message}` }]);
  } finally {
    setLoading(false);
  }
};

const handleKey = (e) => {
  if (e.key === "Enter" && !e.shiftKey) { e.preventDefault(); send(); }
};

```

```

const isEmpty = messages.length === 0 && !loading;

return (
  <div style={{ height:"100vh", display:"flex", flexDirection:"column", backgro
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Syne:wght@800&displ
      @keyframes fadeSlide { from{opacity:0;transform:translateY(10px)} to{opac
      @keyframes pulse { 0%,100%{opacity:.4} 50%{opacity:1} }
      ::-webkit-scrollbar{width:4px} ::-webkit-scrollbar-track{background:#0707
      textarea::placeholder{color:#2d2d50 !important}
      .suggest-btn:hover{border-color:#7c6aff55 !important;color:#c4b5fd !impor
      .new-chat-btn:hover{color:#7c6aff !important}
    `}</style>

    /* Header */
    <div style={{ display:"flex", alignItems:"center", gap:14, padding:"16px 24
      <div style={{ width:40, height:40, background:"linear-gradient(135deg,#0e
      <div>
        <div style={{ fontFamily:"'Syne',sans-serif", fontSize:18, fontWeight:8
        <div style={{ fontSize:10, color:"#3d3d60", letterSpacing:"0.15em" }}>E
      </div>
      <div style={{ marginLeft:"auto", display:"flex", alignItems:"center", gap
        <div style={{ width:7, height:7, borderRadius:"50%", background:"#22c55
        <span style={{ fontSize:11, color:"#3d3d60" }}>ONLINE</span>
      </div>
    </div>

    /* Messages */
    <div style={{ flex:1, overflowY:"auto", padding:"24px 20px", display:"flex"
      {isEmpty && (
        <div style={{ display:"flex", flexDirection:"column", alignItems:"cente
          <div style={{ textAlign:"center" }}>
            <div style={{ fontSize:44, marginBottom:10 }}>⚡</div>
            <div style={{ fontFamily:"'Syne',sans-serif", fontSize:22, fontWeig
            <div style={{ fontSize:12, color:"#3d3d60" }}>Ask anything about co
          </div>
          <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:10,
            {SUGGESTED.map((s, i) => (
              <button key={i} className="suggest-btn" onClick={() => send(s)} s
                {s}
              </button>
            )))
          </div>
        </div>
      </div>
    </div>
  )}

  {messages.map((m, i) => <Message key={i} role={m.role} parts={parseConten

```

```

    {loading && (
      <div style={{ display:"flex", gap:12, alignItems:"flex-start" }}>
        <div style={{ width:34, height:34, borderRadius:"50%", flexShrink:0,
          <div style={{ background:"#0c0c1e", border:"1px solid #1a1a35", borde
            {[0,0.2,0.4].map((d,i) => <div key={i} style={{ width:7, height:7,
              </div>
            </div>
          </div>
        </div>
      <div ref={bottomRef} />
    </div>

    {/ * Input */}
    <div style={{ padding:"14px 20px 18px", borderTop:"1px solid #12122a", back
      {messages.length > 0 && (
        <button className="new-chat-btn" onClick={() => setMessages([])} style=
          ⌚ NEW CHAT
        </button>
      </div>
    <div style={{ display:"flex", gap:10, alignItems:"flex-end", background:"
      <textarea
        ref={textareaRef}
        value={input}
        onChange={e => { setInput(e.target.value); autoResize(); }}
        onKeyDown={handleKey}
        placeholder="Ask a coding question... (Enter to send)"
        rows={1}
        style={{ flex:1, background:"none", border:"none", outline:"none", co
      />
      <button
        onClick={() => send()}
        disabled={loading || !input.trim()}
        style={{ width:36, height:36, borderRadius:9, background:(loading||!i
        >⌚</button>
      </div>
      <div style={{ textAlign:"center", marginTop:8, fontSize:10, color:"#1e1e3
    </div>
  </div>
);
}

```